МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ “ЛЬВІВСЬКА ПОЛІТЕХНІКА”
Інститут Комп’ютерних Наук та Інформатики
Кафедра Систем Штучного Інтелекту

Лабораторна робота №4 з дисципліни
«Опрацювання зображень методами штучного інтелекту»

Виконав:
студ. гр. КН–414
Говдиш Н. О.

Прийняв:
асистент каф. СШІ
Горун. П. П.

Львів – 2024 р.

Тема: детекція об’єктів.
Мета: навчитись вирішувати задачу детекутвання об’єктів.
Варіант: 7
Завдання: На базі OPEN IMAGES DATASET (https://storage.googleapis.com/openimages/web/download.html) реалізувати детекцію об’єктів за допомогою FAST R-CNN
Код:

```
import torch
import torchvision
import fiftyone as fo
import fiftyone.zoo as foz
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor
from torchvision.models.detection import FasterRCNN
from PIL import Image
import torchvision.transforms as T
import numpy as np

dataset = foz.load_zoo_dataset(
    "open-images-v6",
    split="validation",
    max_samples=500,
    label_types=["detections"],
    classes=["Bicycle wheel"],
    bounding_box=True
)

transform = T.Compose([
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])


def load_transform_image(image_path):
    image = Image.open(image_path).convert("RGB")
    return transform(image)


def detections_to_tensors(detections, image_width, image_height):
    boxes = []
    labels = []
    for detection in detections:
        xmin = detection.bounding_box[0] * image_width
        ymin = detection.bounding_box[1] * image_height
        xmax = xmin + (detection.bounding_box[2] * image_width)
        ymax = ymin + (detection.bounding_box[3] * image_height)
```

```
        boxes.append([xmin, ymin, xmax, ymax])
        labels.append(1)

    boxes = torch.as_tensor(boxes, dtype=torch.float32)
    labels = torch.as_tensor(labels, dtype=torch.int64)

    return {"boxes": boxes, "labels": labels}


def create_batches(dataset, batch_size):
    batched_data = []
    for sample in dataset:
        image_path = sample.filepath
        image = load_transform_image(image_path)
        width, height = image.shape[1], image.shape[2]
        target = detections_to_tensors(sample.ground_truth.detections, width, height)
        batched_data.append((image, target))

        if len(batched_data) == batch_size:
            yield batched_data
            batched_data = []

    if batched_data:
        yield batched_data


model = torchvision.models.detection.fasterrcnn_resnet50_fpn(pretrained=True)
num_classes = 2
in_features = model.roi_heads.box_predictor.cls_score.in_features
model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)

device = torch.device('cuda') if torch.cuda.is_available() else torch.device('cpu')
model.to(device)

model.train()
optimizer = torch.optim.SGD(model.parameters(), lr=0.005, momentum=0.9, weight_decay=0.0005)
batch_size = 2
num_epochs = 5

for epoch in range(num_epochs):
```

```
    for batched_images_and_targets in create_batches(dataset, batch_size):
        images = [img.to(device) for img, _ in batched_images_and_targets]
        targets = [{k: v.to(device) for k, v in t.items()} for _, t in batched_images_and_targets]

        loss_dict = model(images, targets)
        losses = sum(loss for loss in loss_dict.values())

        optimizer.zero_grad()
        losses.backward()
        optimizer.step()

    print(f"Epoch #{epoch} loss: {losses.item()}")
```

```python
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from PIL import Image
import torchvision.transforms.functional as F
import torch


def load_image_tensor(image_path, device):
    image = Image.open(image_path).convert("RGB")
    image_tensor = F.to_tensor(image).unsqueeze(0).to(device)
    return image_tensor


def predict_image(model, image_tensor):
    model.eval()
    with torch.no_grad():
        prediction = model(image_tensor)
    return prediction


def visualize_output(image_path, predictions, confidence_threshold=0.5):
    image = Image.open(image_path)
    plt.figure(figsize=(12, 8))
    plt.imshow(image)
    ax = plt.gca()

    if predictions and 'boxes' in predictions:
        boxes = predictions['boxes'].detach().cpu().numpy()
        labels = predictions['labels'].detach().cpu().numpy()
        scores = predictions['scores'].detach().cpu().numpy()

        for box, label, score in zip(boxes, labels, scores):
            if score > confidence_threshold:
                x1, y1, x2, y2 = box
                rect = patches.Rectangle((x1, y1), x2 - x1, y2 - y1, linewidth=2, edgecolor='r', facecolor='none')
                ax.add_patch(rect)
                plt.text(x1, y1, f'{label}: {score:.2f}', bbox=dict(facecolor='yellow', alpha=0.5))

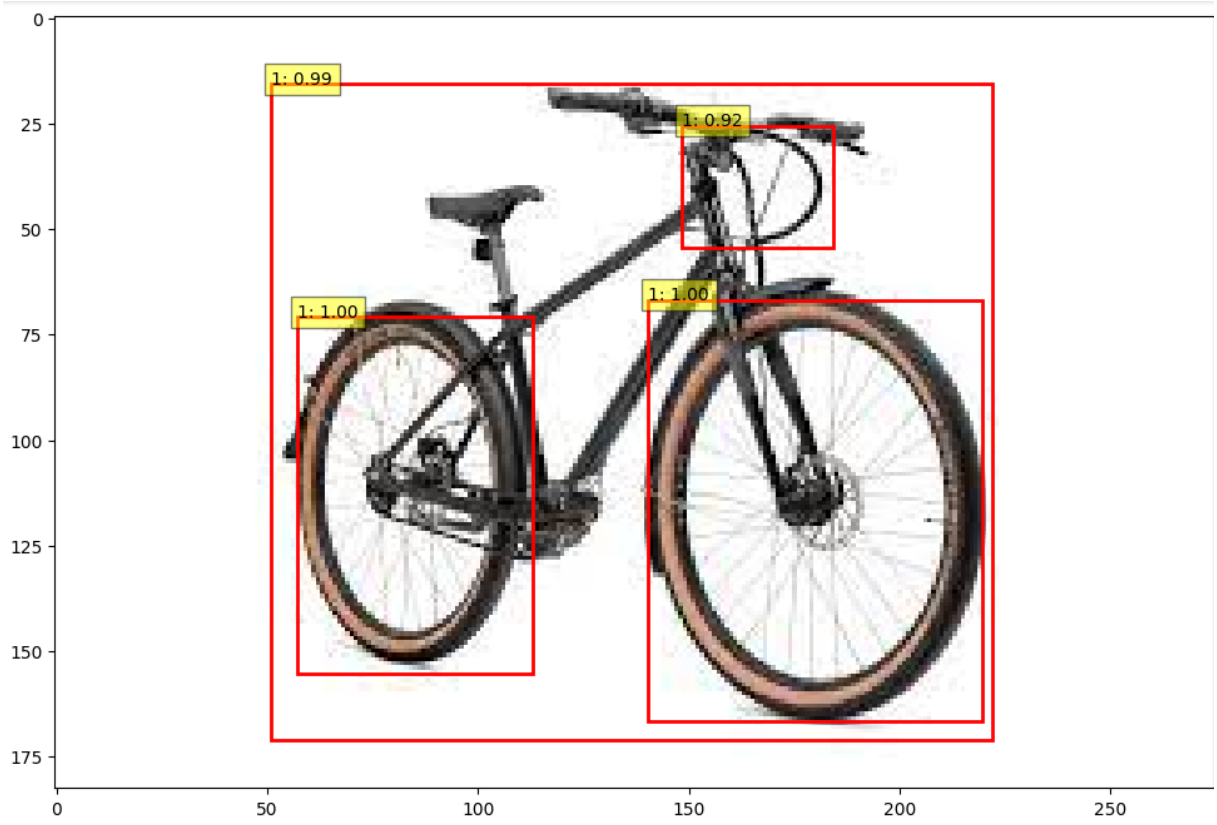
    plt.axis('on')
    plt.show()

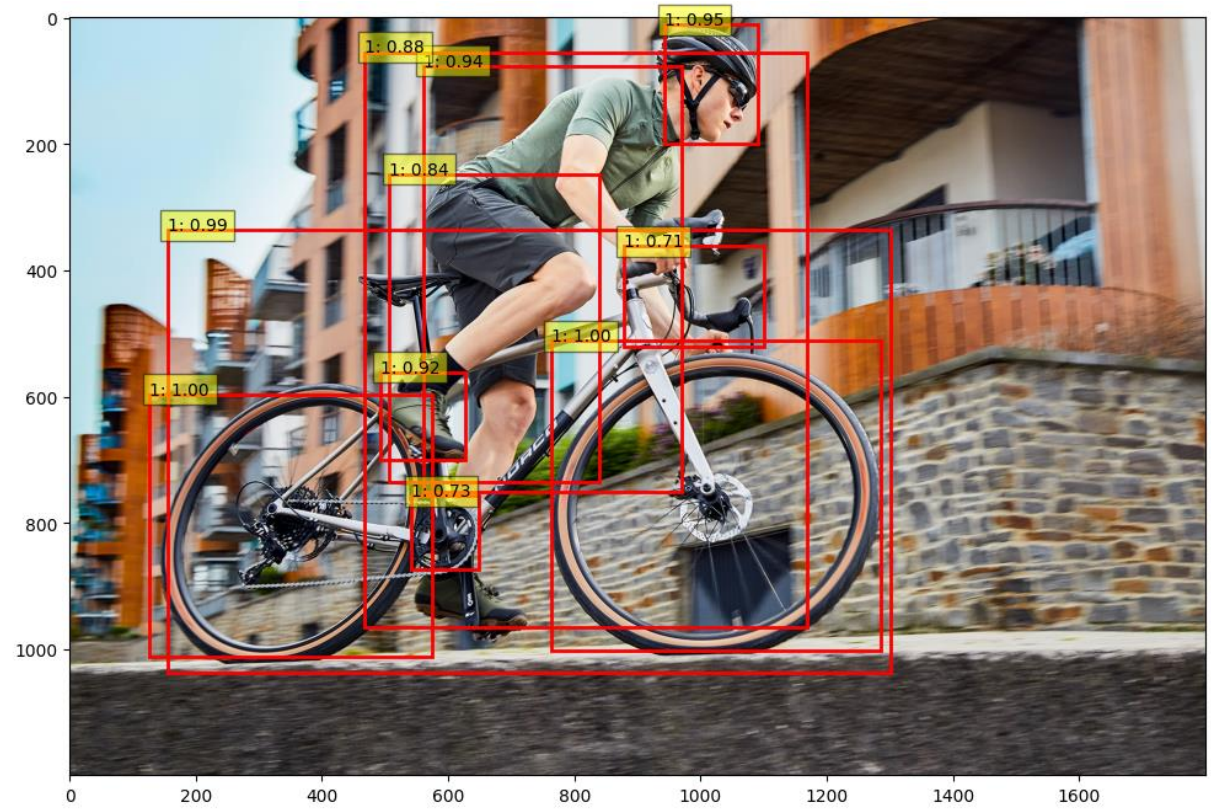
device = torch.device('cuda') if torch.cuda.is_available() else torch.device('cpu')

image_paths = ['/content/drive/MyDrive/img.jpg', '/content/drive/MyDrive/img1.jpg']

for image_path in image_paths:
    image_tensor = load_image_tensor(image_path, device)
    predictions = predict_image(model, image_tensor)
    visualize_output(image_path, predictions[0], confidence_threshold=0.5)
```

Результати:






```
loss: 0.28782862424850464
```

Висновок:

Під час виконання лабораторної роботи я навчився вирішувати задачу детекутвання об’єктів.